Universidad de Buenos Aires

Facultad de Ingeniería

Redes Neuronales (86.54)

Trabajo Practico N°2

2°C 2024

Alumno: Milidoni, Cristian Julian

Padrón: 107647

Noviembre 2024

Contents

1 Ejercicio 1 **2**
1.1 Enunciado . . . 2
1.2 Perceptrón Simple . . . 2
1.2.1 Ecuación del Perceptrón . . . 2
1.2.2 Recta Discriminadora . . . 2
1.2.3 Regla de Actualización de Pesos . . . 2
1.2.4 Error Cuadrático . . . 3
1.3 Implementación y resultados . . . 3

2 Ejercicio 2 **6**
2.1 Enunciado . . . 6
2.2 Implementación y resultados . . . 6

3 Ejercicio 3 **8**
3.1 Enunciado . . . 8
3.2 Perceptrón multicapa . . . 8
3.2.1 Arquitectura de la Red Neuronal . . . 8
3.2.2 Propagación hacia Adelante . . . 8
3.2.3 Error y Retropropagación . . . 8
3.2.4 Entrenamiento . . . 9
3.3 Implementación y resultados . . . 10

4 Ejercicio 4 **13**
4.1 Enunciado . . . 13
4.2 Diferencia entre la actualización en batch y en minibatch . . . 13
4.2.1 Actualización en batch . . . 13
4.2.2 Actualización en minibatch . . . 13
4.3 Inciso A . . . 14
4.4 Inciso B . . . 17

5 Ejercicio 5 **20**
5.1 Enunciado . . . 20
5.2 Máquina de Boltzmann Restringida . . . 20
5.2.1 Entrenamiento . . . 20
5.2.2 Evaluación . . . 21
5.2.3 La base de datos MNIST . . . 21
5.3 Implementación y resultados . . . 21

6 Ejercicio 6 **23**
6.1 Enunciado . . . 23
6.2 Redes Neuronales Convolucionales (CNN) . . . 23
6.2.1 Estructura y Funcionamiento . . . 23
6.2.2 Conceptos Clave . . . 23
6.3 Implementación y resultados . . . 23
6.3.1 Red Neuronal Convolucional . . . 23
6.3.2 Perceptrón multicapa . . . 25

7 Ejercicio 7 **26**
7.1 Enunciado . . . 26
7.2 Autoencoder . . . 26
7.3 Implementación y resultados . . . 26

1 Ejercicio 1

1.1 Enunciado

Implemente un perceptrón simple que aprenda la función lógica AND y la función lógica OR, de 2 y de 4 entradas. Muestre la evolución del error durante el entrenamiento. Para el caso de 2 dimensiones, grafique la recta discriminadora y todos los vectores de entrada de la red.

1.2 Perceptrón Simple

1.2.1 Ecuación del Perceptrón

El perceptrón toma un vector de entrada $\mathbf{x}$ y calcula una salida $\hat{y}$ basada en una combinación lineal de las entradas ponderadas:

$$h(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$$

donde:

- $\mathbf{w} = [w_1, w_2, \ldots, w_n]$ son los pesos asociados a cada entrada.
- b es el término de sesgo (bias).
- $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ son las entradas.

La salida del perceptrón $\hat{y}$ es una función de activación escalonada definida como:

$$\hat{y} = \begin{cases} 1 & \text{si } h(\mathbf{x}) \geq 0 \\ -1 & \text{si } h(\mathbf{x}) < 0 \end{cases}$$

1.2.2 Recta Discriminadora

La recta discriminadora define la frontera entre las dos clases $y = 1$ y $y = -1$. Esta frontera se calcula como:

$$h(\mathbf{x}) = 0 \implies \mathbf{w} \cdot \mathbf{x} + b = 0$$

Para el caso de dos dimensiones ($n = 2$):

$$w_1 x_1 + w_2 x_2 + b = 0 \implies x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$$

Esta ecuación describe una línea recta en el plano x_1-x_2 con pendiente $-\frac{w_1}{w_2}$ y ordenada al origen en $-\frac{b}{w_2}$.

La recta discriminadora separa visualmente las dos clases en el espacio de entrada.

1.2.3 Regla de Actualización de Pesos

Durante el entrenamiento, los pesos $\mathbf{w}$ y el bias b se actualizan en cada epoca para reducir el error. La regla de aprendizaje es:

$$w_i \leftarrow w_i + \eta\, \delta\, x_i \quad \text{para } i = 1, 2, \ldots, n$$
$$b \leftarrow b + \eta\, \delta$$

donde:

- η es la tasa de aprendizaje.
- $\delta = y - \hat{y}$ es el error para un ejemplo dado.

- x_i son los valores de entrada.

En cada época, los patrones de entrada varían su orden para agregarle aleatoriedad al entrenamiento.

1.2.4 Error Cuadrático

El error total en cada época es la suma de los errores cuadrados de todas las muestras de entrenamiento:

$$E = \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

donde:

- m es el número total de muestras de entrenamiento.
- y_i es la etiqueta verdadera de la muestra i.
- $\hat{y}_i$ es la predicción del perceptrón para la muestra i.

Si el error E se reduce a cero, significa que el perceptrón ha aprendido perfectamente las clases en el conjunto de entrenamiento.

1.3 Implementación y resultados

Las funciones a enseñarle al perceptron simple son las funciones logicas OR y AND de 2 y 4 entradas.

A	B	AND
0	0	0
0	1	0
1	0	0
1	1	1

Table 1: Tabla de verdad compuerta AND para 2 entradas

A	B	OR
0	0	0
0	1	1
1	0	1
1	1	1

Table 2: Tabla de verdad compuerta OR para 2 entradas

A	B	C	D	AND
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

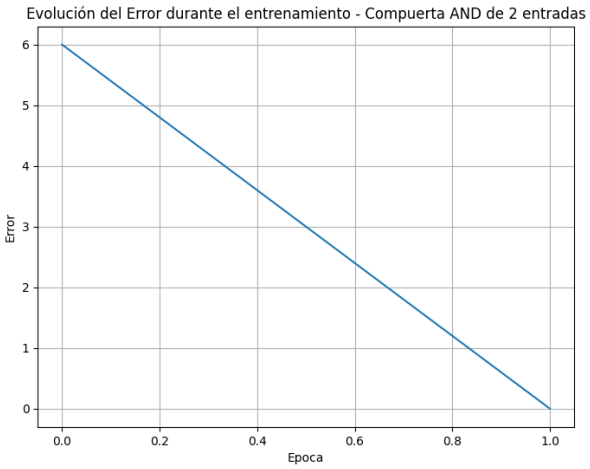
Table 3: Tabla de verdad de la compuerta AND para 4 entradas

A	B	C	D	OR
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

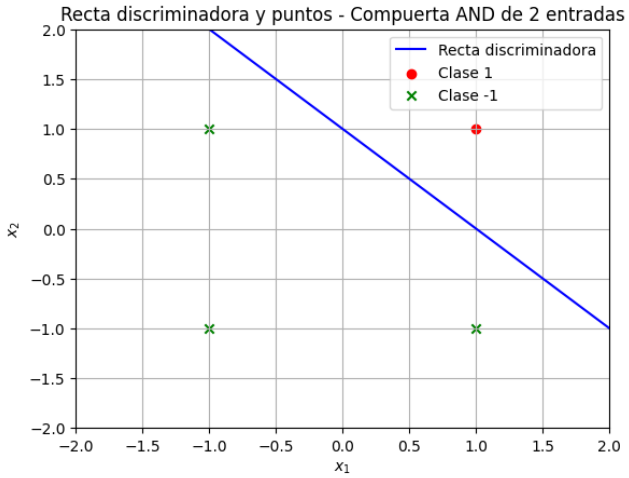
Table 4: Tabla de verdad de la compuerta OR para 4 entradas

Se entrena al perceptrón simple con una tasa de aprendizaje de 0.1 y reemplazando las entradas 0 por -1, dado que la función de activación es la función signo. Para el caso de la función OR y AND de 2 entradas, obtenemos la recta

discriminatoria y el error en función de la época y para el caso de la función OR y AND de 4 entradas, obtenemos solo el error en función de la época.

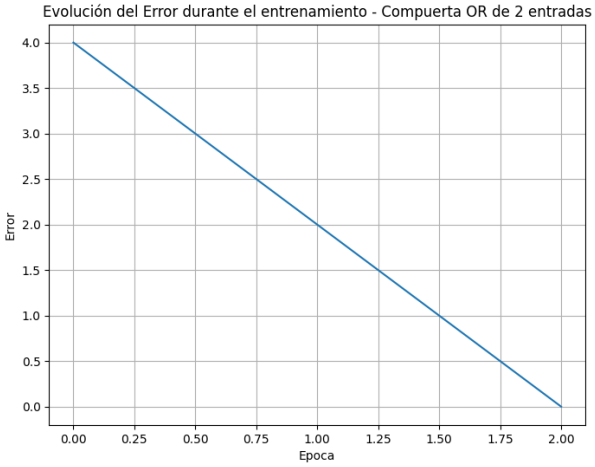


(a) Evolución del error durante el entrenamiento

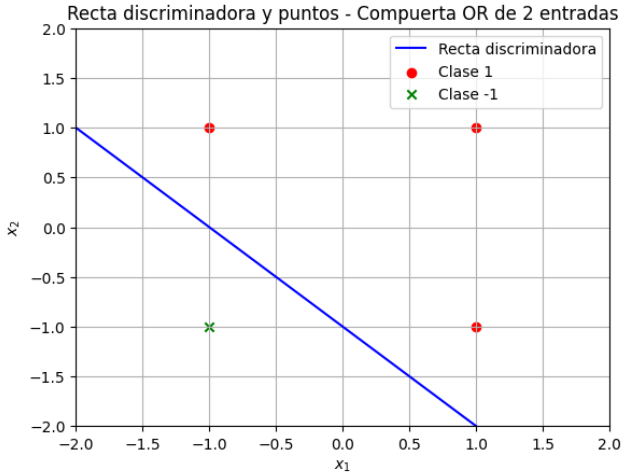


(b) Recta discriminadora

Figure 1: Compuerta AND de 2 entradas



(a) Evolución del error durante el entrenamiento



(b) Recta discriminadora

Figure 2: Compuerta OR de 2 entradas

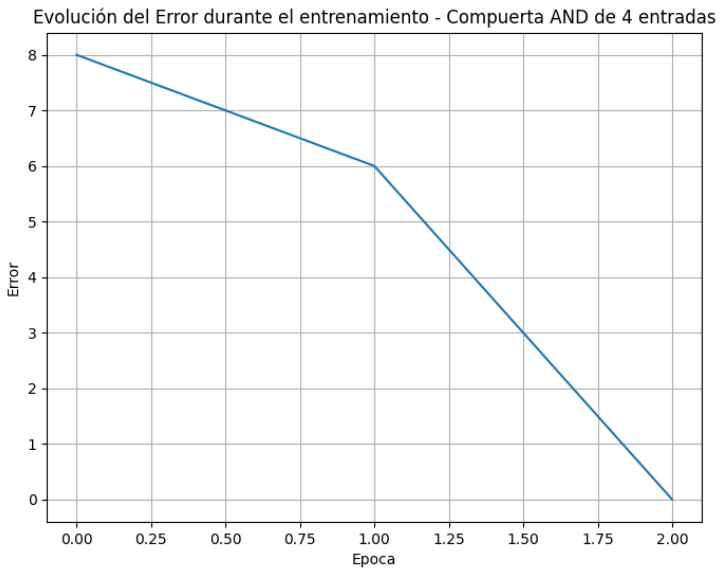


Figure 3: Evolución del error durante el entrenamiento - Compuerta AND de 4 entradas

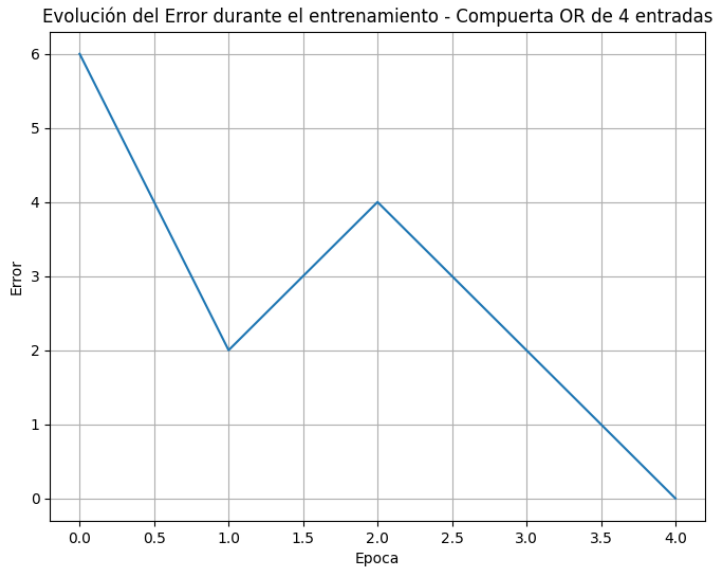


Figure 4: Evolución del error durante el entrenamiento - Compuerta OR de 4 entradas

Se observa que en la Fig. 1 y 2 la recta discriminadora logra diferenciar perfectamente entre ambas clases de resultados. Además, a partir de varios experimentos, se determino que la función que mas épocas necesita para aprender el perceptrón simple es la OR de 4 entradas siendo como máximo 5 épocas según el orden de los patrones en el entrenamiento.

2 Ejercicio 2

2.1 Enunciado

Determine numéricamente cómo varía la capacidad del perceptrón simple en función del número de patrones enseñados.

2.2 Implementación y resultados

Se define la capacidad del perceptrón simple como:

$$C(N, N_p) = \frac{\#\text{ Problemas linealmente separables}}{\#\text{ Problemas Totales}}$$

donde:

- N Cantidad de features o dimensión del espacio de patrones
- N_p Cantidad de patrones a enseñar.

El algoritmo para estimar de forma numérica la capacidad del perceptrón simple en función del número de patrones enseñados para un N fijo:

1. Generar N_p patrones: $\vec{X}_n(i) \sim \mathcal{U}(-1, 1)$ y $Y_n \sim \mathcal{B}(p = \frac{1}{2})$
2. Entrenar el perceptrón
3. Si el perceptrón aprende el problema: $N_p \longleftarrow N_p + 1$

Básicamente se busca ver cuantos problemas de una determinada cantidad N_p de patrones puede aprender el perceptron simple. Es decir, al perceptron simple se le presentan siempre 1000 problemas de un determinado N_p número de patrones. Para cada N_p, observamos cuantos de estos 1000 problemas fue capaz de aprender (el error se redujo a 0). El entrenamiento fue siempre de 50 épocas con una tasa de aprendizaje de 0.1. En las Fig. 5, 6 y 7 se grafican las capacidades numéricas calculadas para una dimensión del espacio de patrones N de 3, 4 y 5. Además, en cada caso se grafica la capacidad teórica que equivale a un escalón en $2\,N$.

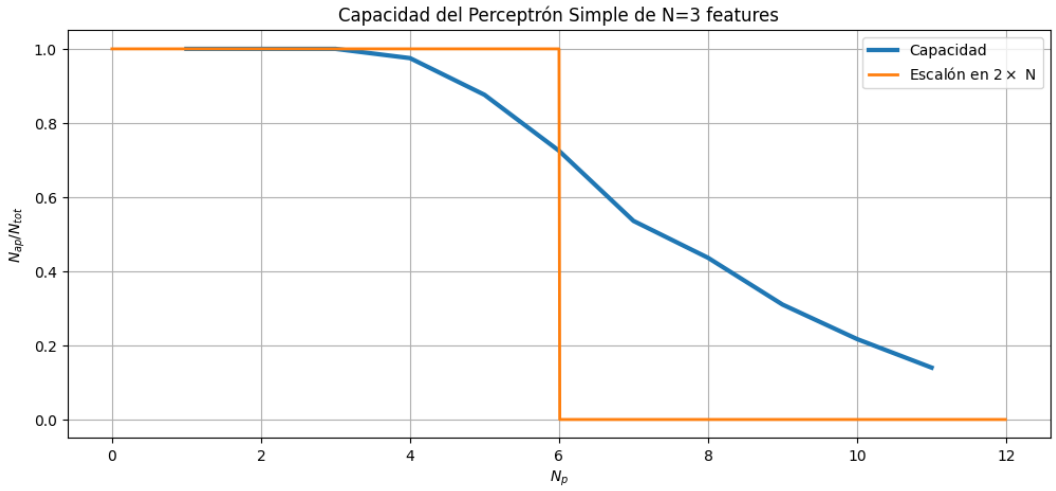


Figure 5: Capacidad del Perceptrón Simple N=3

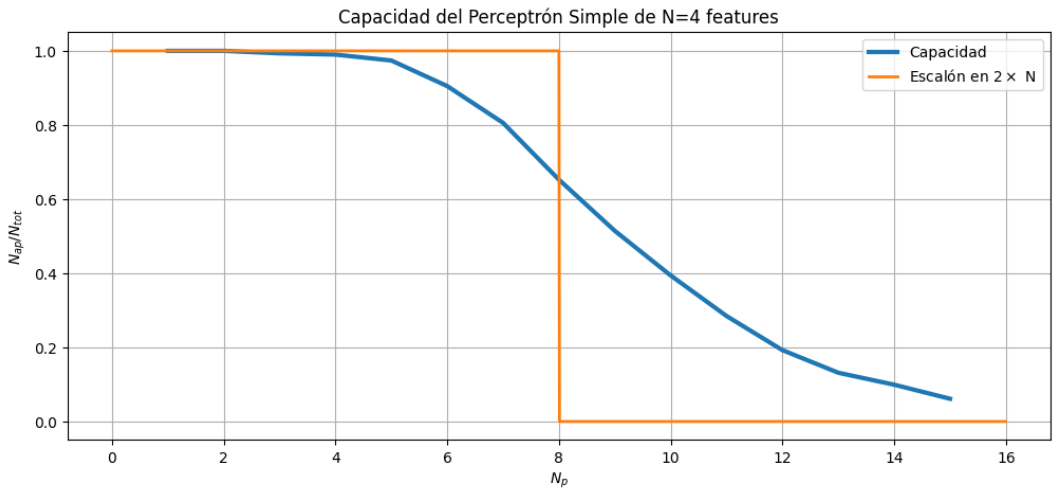


Figure 6: Capacidad del Perceptrón Simple N=4

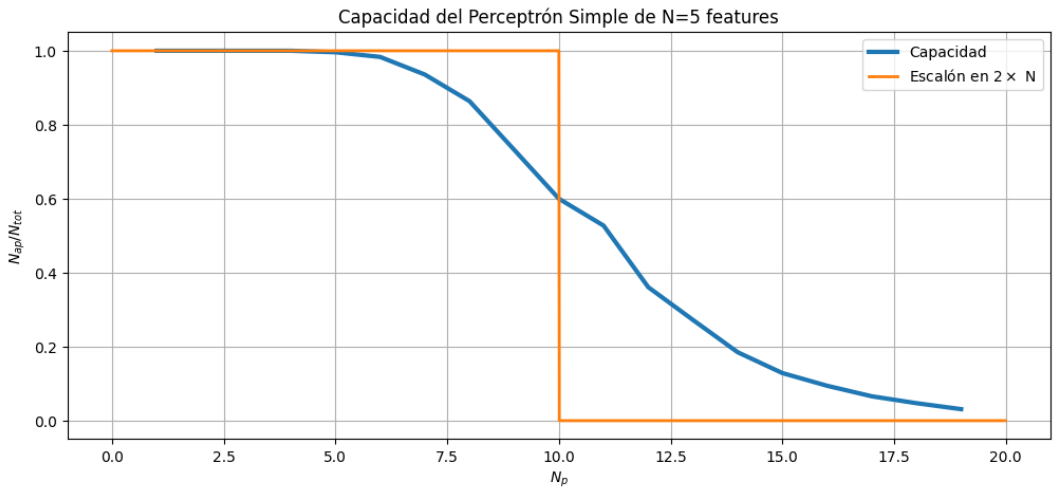


Figure 7: Capacidad del Perceptrón Simple N=5

3 Ejercicio 3

3.1 Enunciado

Implemente un perceptrón multicapa que aprenda la función lógica XOR de 2 y de 4 entradas (utilizando el algoritmo Backpropagation y actualizando en batch). Muestre cómo evoluciona el error durante el entrenamiento.

3.2 Perceptrón multicapa

3.2.1 Arquitectura de la Red Neuronal

Para un perceptrón multicapa, la salida x_i de la neurona i está determinada por la entrada ponderada h_i, que se calcula como:

$$h_i = \sum_j w_{ij} x_j + b_i$$

donde:

- w_{ij} es el peso que conecta la neurona j de la capa anterior con la neurona i de la capa actual.
- x_j es la salida de la neurona j en la capa anterior.
- b_i es el sesgo de la neurona i en la capa actual.

Luego, la salida x_i de la neurona i se obtiene aplicando una función de activación g:

$$x_i = g\left(h_i\right)$$

La función de activación que se usara es la tangente hiperbólica $g(x)$ se define como:

$$g(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Y su derivada $g'(x)$ es:

$$g'(x) = 1 - \tanh^2(x)$$

3.2.2 Propagación hacia Adelante

La propagación hacia adelante consiste en calcular las activaciones de todas las neuronas en cada capa. En términos generales, para una capa:

$$h_i = \sum_j w_{ij} x_j + b_i$$
$$x_i = g\left(h_i\right)$$

3.2.3 Error y Retropropagación

El error en la capa de salida se define como E y se calcula como:

$$E = \frac{1}{2} \sum_\mu \left(y_d^\mu - y^\mu\right)^2$$

donde y_d^μ es la salida deseada de la red e y^μ es la salida obtenida para el patrón μ.

Luego, definimos la variación de los pesos sinápticos usando la propagación hacia atrás de los errores.

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$

Si bien el error se define para todos los patrones μ evaluados en cada época, buscaremos minimizar el error por cada patrón. Luego, a partir de esto se calculara un Δw_{ij}^{μ} para cada patrón y se actualizaran los pesos sinápticos a partir de la suma de los mismos.

Entonces, tomando la derivada del error respecto a un solo patrón:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial h_i}{\partial w_{ij}} \frac{\partial E}{\partial h_i} = x_j \, \delta_i$$

Si la neurona i pertenece a la capa de salida, vale que:

$$\delta_i = (y_d - y) \cdot g'(h_i)$$

Si la neurona i pertenece a capas intermedias, el δ_i se calcula mediante una sumatoria sobre las neuronas de la capa k siguiente:

$$\delta_i = g'(h_i) \sum_k w_{ik} \delta_k$$

Por lo tanto, la variación de los pesos sinápticos para cada patrón u es:

$$\Delta w_{ij}^{\mu} = -\eta \, x_j \, \delta_i$$

Los pesos se actualizan utilizando el gradiente descendente. La actualización de los pesos w_{ij} en cada época es:

$$w_{ij} := w_{ij} + \sum_{\mu} \Delta w_{ij}^{\mu}$$

Para los sesgos b_i, la actualización es:

$$b_i := b_i - \eta \sum_{\mu} \delta_i^{\mu}$$

3.2.4 Entrenamiento

El proceso de entrenamiento consta de las siguientes etapas:

1. **Propagación hacia adelante**: calcular las activaciones x_i^{μ} de cada neurona para cada patrón μ.
2. **Cálculo del error**: calcular el error E^{μ} y sus gradientes δ_i^{μ} para cada patrón μ.
3. **Retropropagación del error**: actualizar los errores δ_i^{μ} de las capas anteriores para cada patrón μ.
4. **Actualización de los pesos y sesgos**: ajustar los pesos y sesgos usando las derivadas, sumando sobre todos los patrones μ.

Para mejorar el rendimiento computacional, todos los cálculos fueron llevados a su equivalente matricial.

3.3 Implementación y resultados

Una vez implementado el perceptrón multicapa aplicando todo lo expuesto en los incisos anteriores, se entrenó un perceptrón multicapa para que aprendiera la XOR de 2 entradas y otro para que aprendiera la XOR de 4 entradas. Para ambos casos se uso la mínima cantidad teórica de neuronas capaz de aprender ambas funciones. En el caso de la XOR de 2 entradas, la capa oculta consta de 3 neuronas mientras que en el caso de la XOR de 4 entradas, la capa oculta consta de 6 neuronas. En ambos casos, la tasa de aprendizaje utilizada fue de 0.01.

Las funciones lógicas XOR de 2 y 4 entradas se definen como:

Entrada 1	**Entrada 2**	**Resultado Deseado**
0	0	0
0	1	1
1	0	1
1	1	0

Table 5: Tabla de Verdad de la XOR de 2 Entradas

Entrada 1	**Entrada 2**	**Entrada 3**	**Entrada 4**	**Resultado Deseado**
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Table 6: Tabla de Verdad de la XOR de 4 Entradas

Observar que la función XOR es un contador de paridad, la salida es 1 cuando en las entradas hay una cantidad impar de 1s y es 0 cuando hay una cantidad par.

Para facilitar el aprendizaje de la XOR al perceptron multicapa, dado que se usa la función de activación $tanh(x)$, los 0s en las entradas y salidas fueron reemplazados por -1s.

En la Fig. 8 y 9, se ilustra la evolución del error durante el entrenamiento del perceptron multicapa para la XOR de 2 entradas y de 4 entradas, respectivamente.

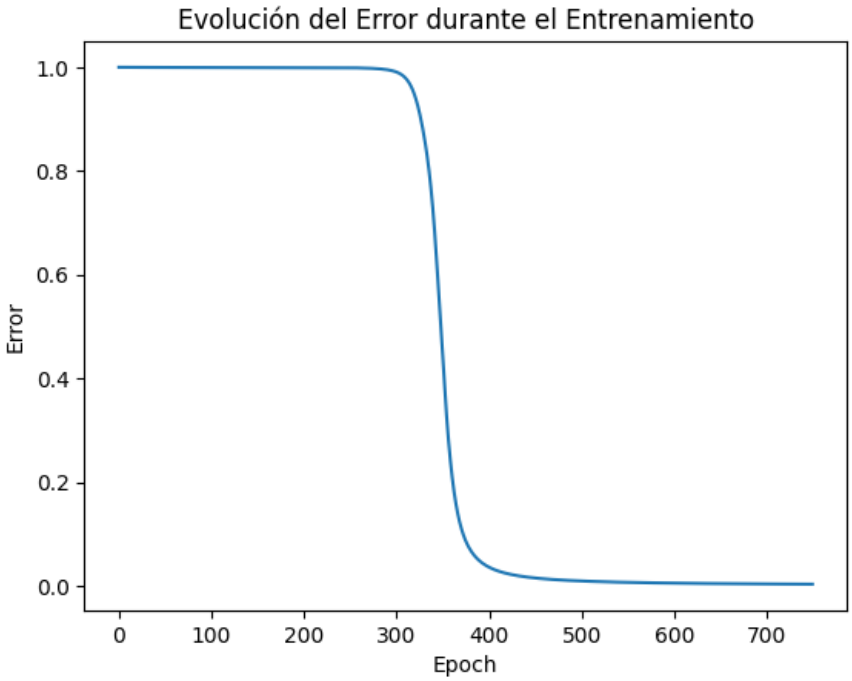


Figure 8: Evolución del error durante el entrenamiento - XOR de 2 entradas

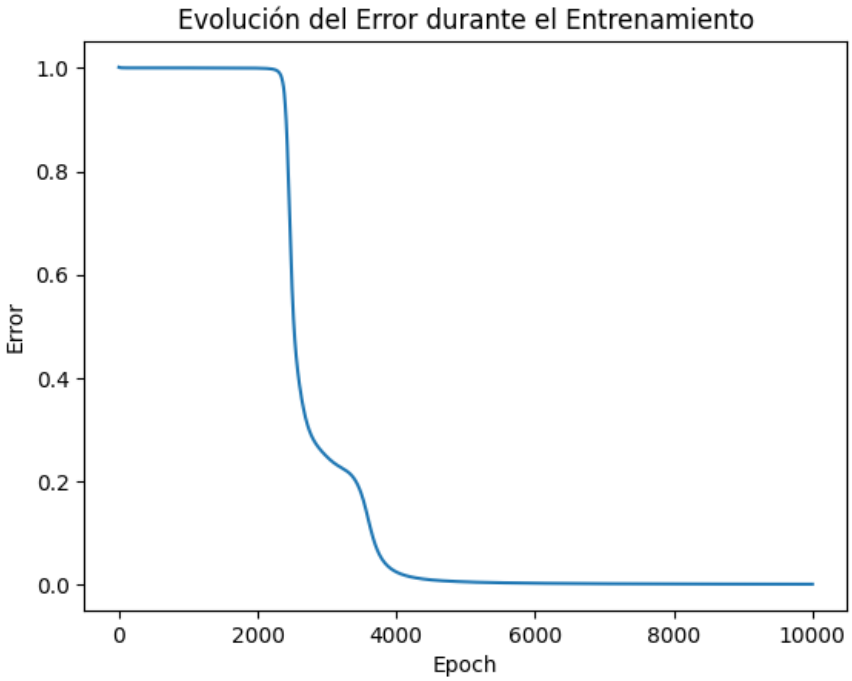


Figure 9: Evolución del error durante el entrenamiento - XOR de 4 entradas

Se observa que el perceptron multicapa logra reducir el error casi a nulo pero nunca será completamente nulo dado que la tangente hiperbólica tiene un asíntota superior en 1 y una asíntota inferior en -1. Cabe resaltar que dado que al entrenar por cada época se mezclan los patrones de entrada, que los pesos sinápticos al inicio del entrenamiento se inicializan aleatoriamente y que se esta utilizando la cantidad mínima de neuronas en la capa oculta que aprenden la

XOR, hubo ocasiones donde el perceptron aprendió la XOR pero requirió de mas épocas de entrenamiento o no logro reducir tanto el error hasta volverlo casi nulo o directamente no consiguió aprender la función.

Finalmente, para estas realizaciones del experimento se muestra los resultados al evaluar la red en los patrones de entrada de la XOR para 2 entradas y para 4 entradas, se comprueba que aprendió los patrones con gran precisión.

Resultado Deseado	**Resultado Obtenido**
-1	-0.95650353
1	0.93883881
1	0.93854757
-1	-0.95614625

Table 7: Evaluación del Modelo para XOR de 2 Entradas

Resultado Deseado	**Resultado Obtenido**
-1	-0.96865116
1	0.99740542
1	0.97628147
-1	-0.97117485
1	0.93344245
-1	-0.96751961
-1	-0.97818465
1	0.97661874
1	0.98862566
-1	-0.98503952
-1	-0.98249913
1	0.95026483
-1	-0.95661035
1	0.98758198
1	0.99902159
-1	-0.98329274

Table 8: Evaluación del Modelo para XOR de 4 Entradas

4 Ejercicio 4

4.1 Enunciado

a) Implemente una red con aprendizaje por *Backpropagation* que aprenda la siguiente función:

$$f(x, y, z) = \sin(x) + \cos(y) + z,$$

donde $x, y \in [0, 2\pi]$ y $z \in [-1, 1]$. Para ello:

- Construya un conjunto de datos de entrenamiento y un conjunto de evaluación.
- Muestre la evolución del error de entrenamiento y de evaluación en función de las épocas de entrenamiento.

b) Estudie la evolución de los errores durante el entrenamiento de una red con una capa oculta de 30 neuronas cuando el conjunto de entrenamiento contiene 40 muestras. Responda:

- ¿Qué ocurre si el minibatch tiene tamaño 40?
- ¿Qué ocurre si el minibatch tiene tamaño 1?

4.2 Diferencia entre la actualización en batch y en minibatch

En el entrenamiento de redes neuronales, la actualización de los parámetros ($\mathbf{W}$, $\mathbf{b}$) se realiza minimizando una función de error, generalmente mediante el descenso del gradiente. Existen dos estrategias principales para calcular el gradiente: la **actualización en batch** y la **actualización en minibatch**, esta última con gradiente estocástico.

4.2.1 Actualización en batch

En la *actualización en batch*, el gradiente se calcula utilizando la totalidad del conjunto de datos de entrenamiento en cada iteración. Esto significa que para cada actualización de los parámetros, la red utiliza todos los patrones disponibles.

En cada epoca se calcula el error de entrenamiento y se actualizan los pesos sinápticos con las siguientes reglas:

$$E = \sum_{\mu} (y_d^{\mu} - y^{\mu})^2$$

$$w_{ij} \longleftarrow w_{ij} + \sum_{\mu} \Delta w_{ij}^{\mu}$$

Siendo μ la totalidad de los patrones del set de entrenamiento.

Entonces el error se computa por época usando la totalidad de los patrones de entrenamiento en cada una.

4.2.2 Actualización en minibatch

La *actualización en minibatch* introduce el concepto de dividir el conjunto de datos de entrenamiento en pequeños subconjuntos denominados *minibatches*. En cada iteración, el gradiente se calcula utilizando únicamente los patrones de un minibatch.

$$E_{train} = \sum\nolimits_{\mu \in \text{minibatches}} \left(y_{d_{train}}^{\mu} - y^{\mu}\right)^2$$

$$w_{ij} \longleftarrow w_{ij} + \sum\nolimits_{\mu \in \text{minibatches}} \Delta w_{ij}^{\mu}$$

Además, al terminar una época, es decir, al recorrer todos los minibatches una vez, se evaluara la red en el set de testeo, definiendo asi un error de testeo por cada época

$$E_{test} = \sum_{\mu} \left(y_{d_{test}}^{\mu} - y^{\mu}\right)^2$$

4.3 Inciso A

Para implementar una red que aprenda la función solicitada utilizando backpropagation, se utilizo un perceptrón multicapa con una capa oculta de 30 neuronas, una constante de aprendizaje $\eta = 0.0007$, y se la entreno por 1500 épocas.

La función de activación utilizada durante el entrenamiento fue la tangente hiperbólica y para el entrenamiento se generaron 500 puntos (x, y, z) de manera aleatoria respetando los rangos de cada variable especificados por el ejercicio.

Cabe resaltar que el entrenamiento se realizo separando los datos de entrada en minibatches de 10 puntos. Además, el set de testeo consto de 200 puntos también generados aleatoriamente. Durante el entrenamiento se calculo el error de entrenamiento por iteración y el error de testeo por época. Finalmente, una vez entrenada la red, se la evaluó en el set de entrenamiento con el fin de ilustrar la dispersión entre la salida esperada de los puntos del set de testeo y la salida obtenida.

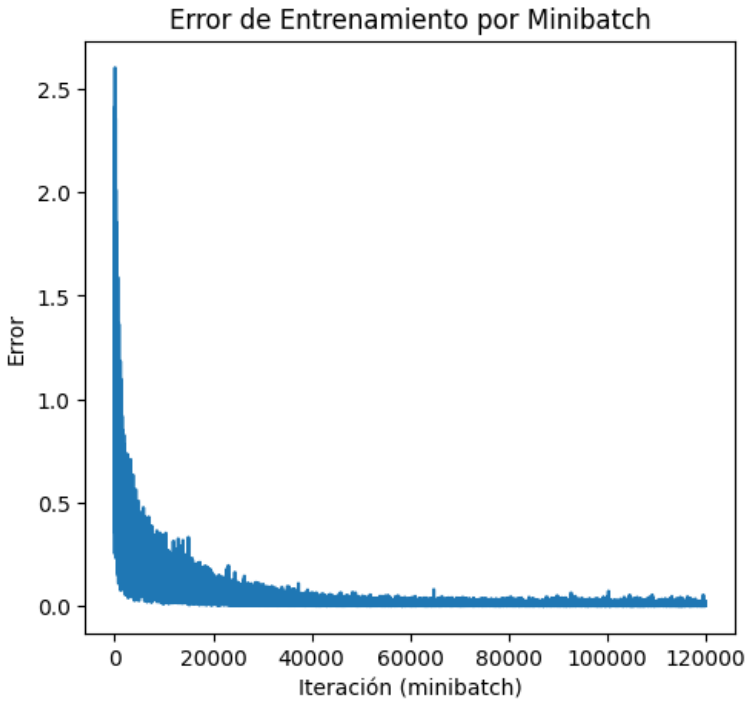


Figure 10: Evolución del error de entrenamiento - Perceptrón Multicapa con capa oculta de 30 neuronas

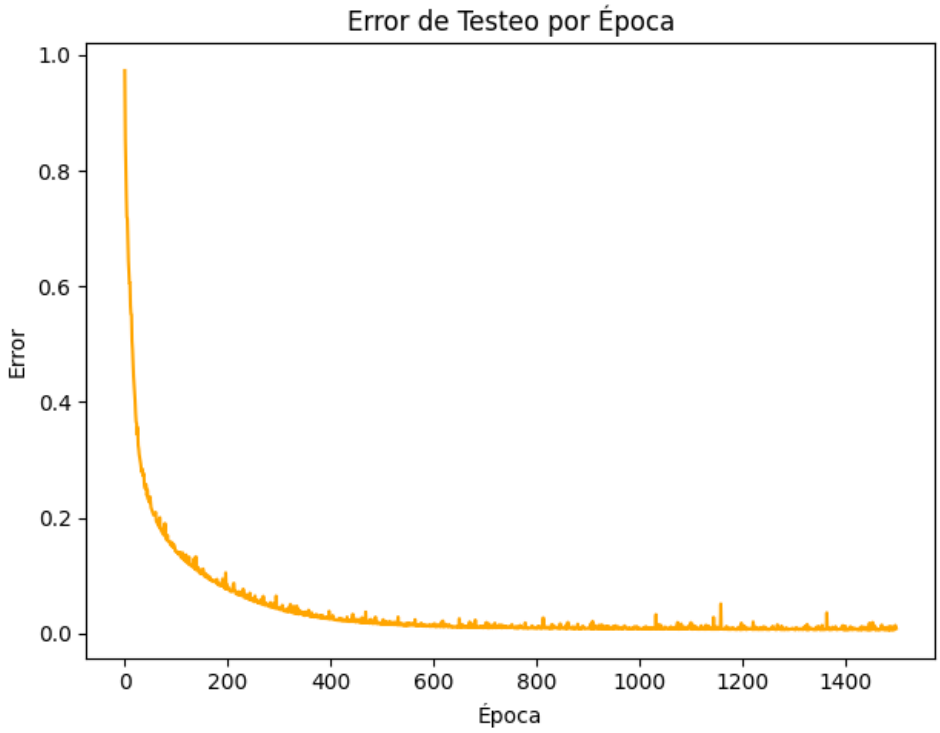


Figure 11: Evolución del error de testeo - Perceptrón Multicapa con capa oculta de 30 neuronas

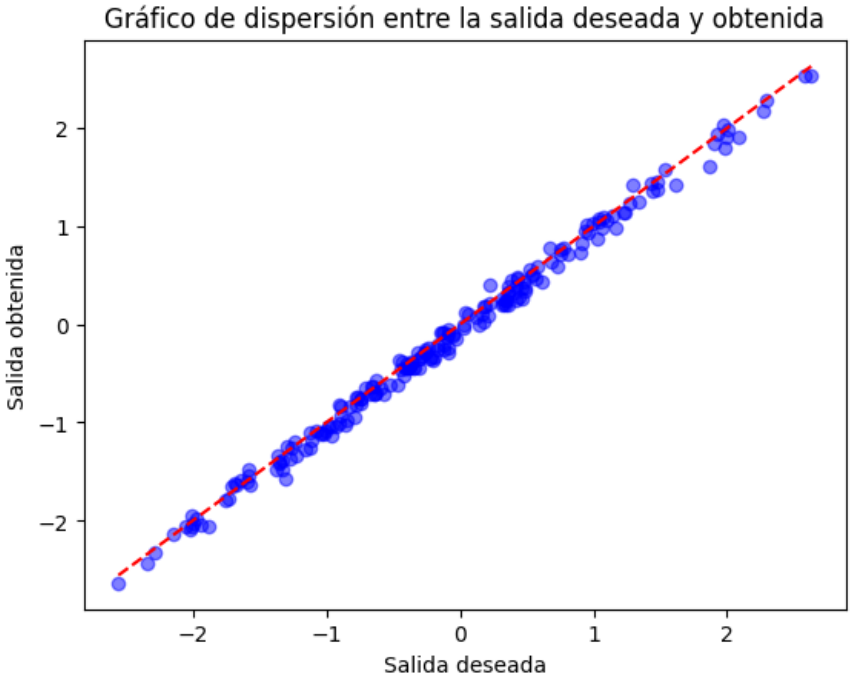


Figure 12: Dispersión de las salidas deseadas del set de entrenamiento con las obtenidas al evaluar la red en el set de testeo

Luego, se repitió el experimento utilizando la misma cantidad de muestras en el set de entrenamiento y en el set de testeo, la misma constante de aprendizaje, el mismo tamaño de minibatches y las misma cantidad de épocas de

entrenamiento pero ahora utilizando un perceptron multicapa con una capa oculta de 5 neuronas.

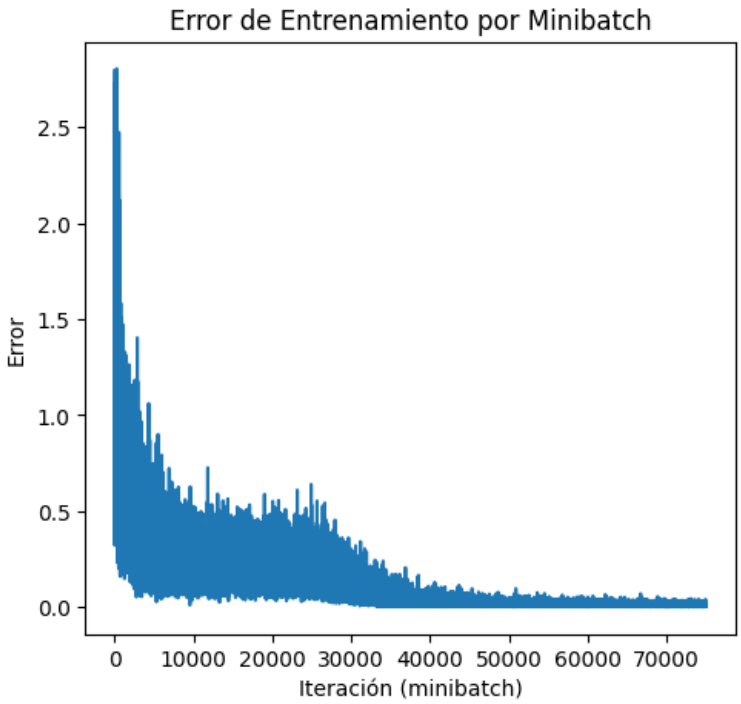


Figure 13: Evolución del error de entrenamiento - Perceptrón Multicapa con capa oculta de 5 neuronas

Figure 14: Evolución del error de testeo - Perceptrón Multicapa con capa oculta de 5 neuronas

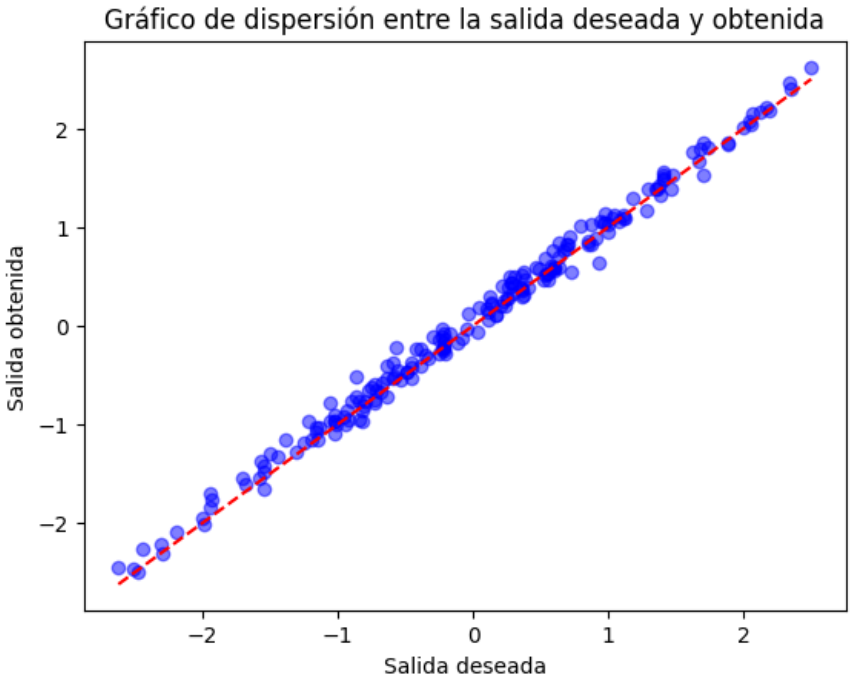


Figure 15: Dispersión de las salidas deseadas del set de entrenamiento con las obtenidas al evaluar la red en el set de testeo

En base al error de testeo y a la dispersión entre la salida obtenida y deseada, se puede afirmar que en ambos casos se obtuvieron resultados muy satisfactorios. Para la época 1000 del entrenamiento, ambos errores de testeo eran similares lo que demuestra que con una cantidad 6 veces menor de neuronas se obtienen resultados igualmente exitosos.

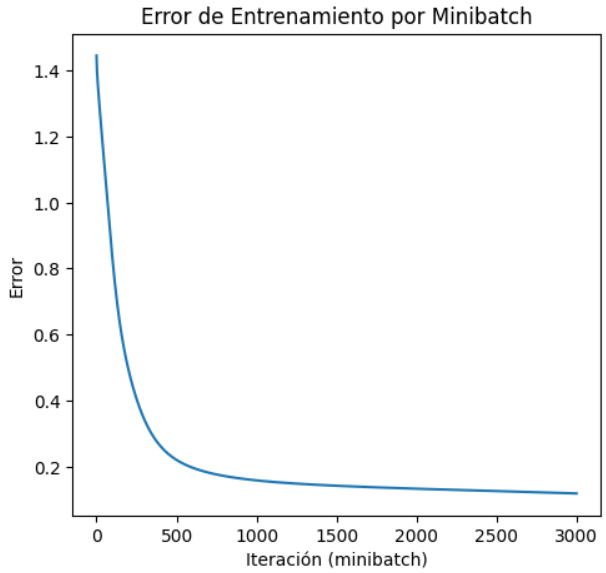
4.4 Inciso B

Para este inciso, se entrenó un perceptrón multicapa con una capa oculta de 30 neuronas y un set de entrenamiento de 40 muestras. En una realización del experimento se utilizó un tamaño de minibatch de 40 y en otra un tamaño de 1.

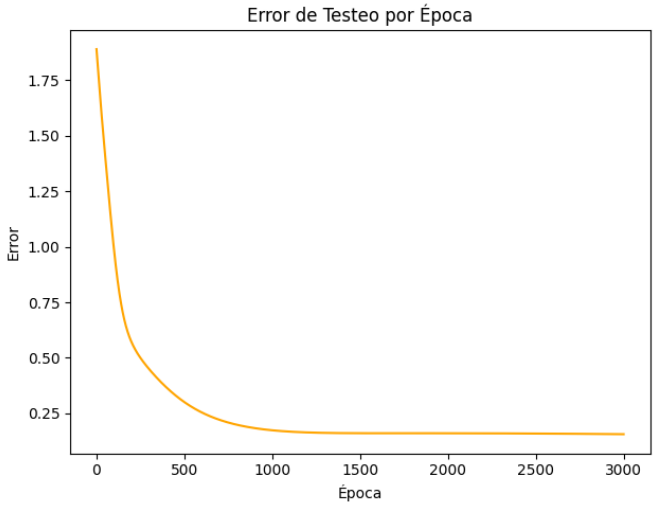
Cuando el tamaño del minibatch es igual al tamaño del set de entrenamiento, el entrenamiento se realiza utilizando todos los datos en cada actualización de los pesos sinápticos. la evolución del error tiende a ser más suave y menos ruidosa porque utiliza toda la información disponible para calcular el gradiente. Las actualizaciones son más estables, pero el modelo puede tardar más en converger porque cada iteración es más costosa computacionalmente. El único problema es que durante el entrenamiento la red puede quedarse atrapado en mínimos locales debido a la ausencia de ruido estocástico.

Otro problema de la ausencia de ruido estocástico es que se puede producir el fenómeno de overfitting en donde el modelo aprende demasiado bien los patrones del conjunto de entrenamiento, incluso aquellos que son específicos o ruidosos, en lugar de generalizar correctamente a datos nuevos no vistos.

En las Fig. 16 y 17 se observa como ocurren ambos fenómenos. En el caso de la Fig. 16, durante el entrenamiento la actualización de los pesos sinápticos se detuvo al alcanzar un mínimo local del cual al entrenar no se pudo volver a salir. En el caso de la Fig. 17 se observa que se logro reducir el error de entrenamiento casi a nulo pero a medida que fue avanzando el entrenamiento, el error de testeo volvió a aumentar por el sobre aprendizaje del set de entrenamiento.



(a) Evolución del error durante el entrenamiento



(b) Evolución del error de testeo

Figure 16: Entrenamiento con minibatch de 40 muestras - mínimo local

(a) Evolución del error de entrenamiento

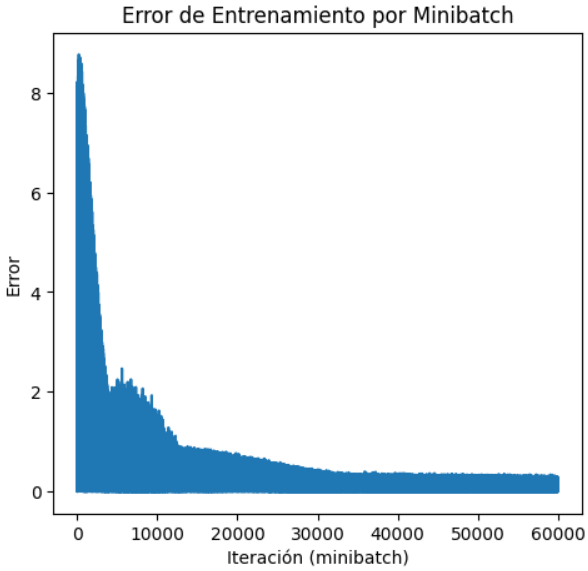
(b) Evolución del error de testeo

Figure 17: Entrenamiento con minibatch de 40 muestras - overfitting

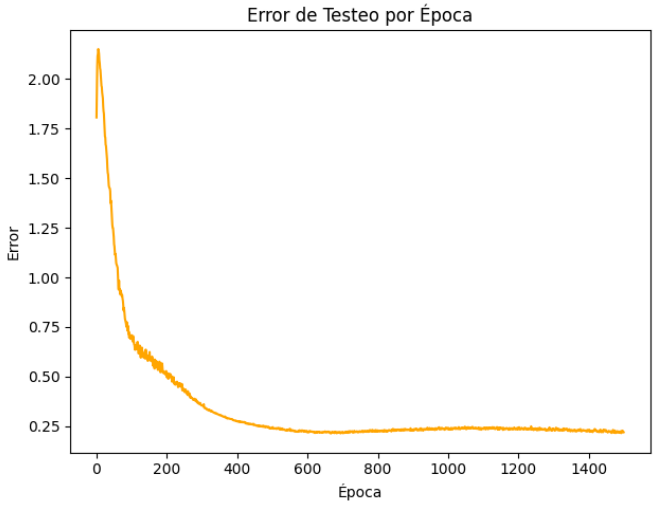
Cuando el minibatch tiene tamaño 1, cada actualización de los pesos se realiza con una sola muestra del conjunto de datos. Esto introduce ruido estocástico en el cálculo del gradiente. La evolución del error es mucho más fluctuante debido a la variabilidad de las muestras. Aunque este ruido puede ayudar a escapar de mínimos locales, puede dificultar la convergencia hacia un mínimo global óptimo. Puede converger más rápido en algunos casos debido a las frecuentes actualizaciones, pero la solución puede ser más ruidosa o subóptima. También se consume menos memoria computacional.

En la Fig. 18 y 19, se observan como los mínimos globales no son los mas óptimos en cuanto al error de testeo pero que se alcanzan a mayor velocidad. También se observa mucho mas ruido en ambos errores y en la Fig 19 se ve como

se logra escapar de un mínimo local gracias al ruido estocástico.

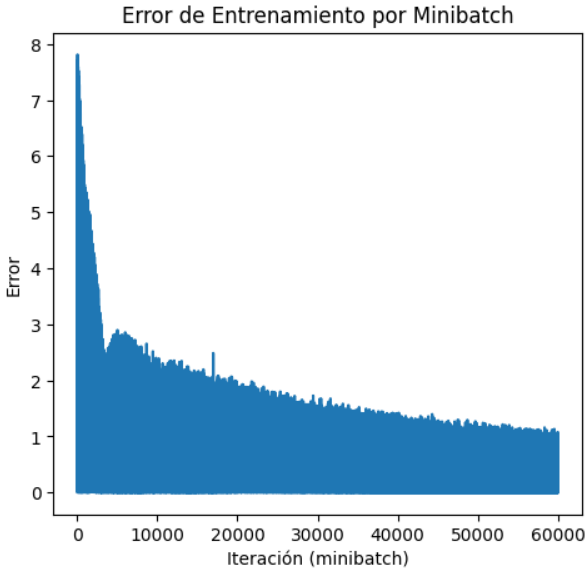


(a) Evolución del error de entrenamiento

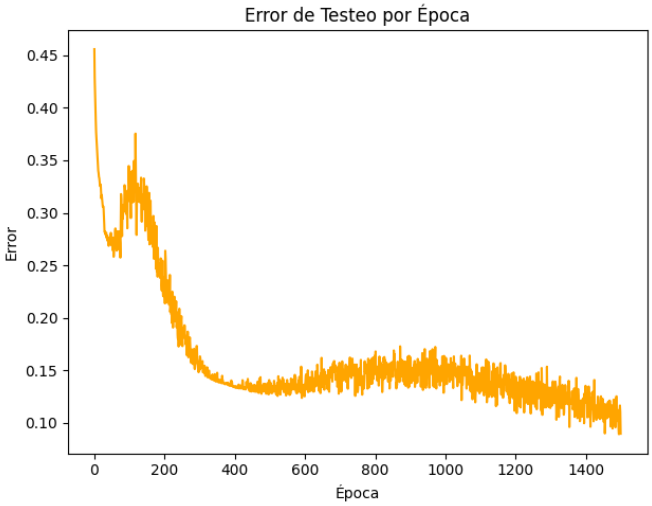


(b) Evolución del error de testeo

Figure 18: Entrenamiento con minibatch de 1 muestra - experimento 1



(a) Evolución del error durante el entrenamiento



(b) Recta discriminadora

Figure 19: Entrenamiento con minibatch de 1 muestra - experimento 2

5 Ejercicio 5

5.1 Enunciado

Siguiendo el trabajo de Hinton y Salakhutdinov (2006), entrenar una máquina restringida de Boltzmann con imágenes de la base de datos MNIST. Se debe mostrar el error de reconstrucción durante el entrenamiento y ejemplos de cada uno de los dígitos reconstruidos.

5.2 Máquina de Boltzmann Restringida

Una Máquina de Boltzmann Restringida (*Restricted Boltzmann Machine*, RBM) es un modelo generativo estocástico utilizado para aprender una representación eficiente de los datos. Es un tipo de red neuronal que combina probabilidades con redes neuronales para modelar distribuciones complejas.

La arquitectura de una RBM consiste en:

- **Capa visible (v):** Representa las variables observables o los datos de entrada. Cada neurona en esta capa corresponde a una característica de los datos.
- **Capa oculta (h):** Contiene neuronas que capturan patrones subyacentes en los datos.
- **Pesos (W):** Matriz de pesos que conecta cada neurona visible con cada neurona oculta. No hay conexiones entre las neuronas de la misma capa, lo que garantiza la restricción *"restringida"*.

5.2.1 Entrenamiento

Dado que los patrones de entrada son imágenes en escala de grises, el algoritmo de entrenamiento para la maquina de Boltzmann es el siguiente:

1) $m_i = \text{pixel}(i)$
2) $v_i \sim \mathcal{N}(\mu = m_i, \sigma = 1)$
3) $h_j = 1$ con probabilidad $p = \sigma\left(\sum_i w_{ij} v_i + b_j\right)$ con $\sigma(x) = \dfrac{1}{1+e^{-x}}$
4) Guardar (v_i, h_j) en el conjunto DATA
5) $m_i = \sum_i w_{ij} h_j + b_i$
6) $v_i \sim \mathcal{N}(\mu = m_i, \sigma = 1)$
7) $h_j = 1$ con probabilidad $p = \sigma\left(\sum_i w_{ij} v_i + b_j\right)$ con $\sigma(x) = \dfrac{1}{1+e^{-x}}$
8) Guardar (v_i, h_j) en el conjunto RECON

Se repiten estos pasos para todas las imágenes del conjunto de entrenamiento o se entrena en *minibatches* como en el ejercicio anterior. Luego, las actualizaciones de pesos sinápticos y sesgos se realizan como:

$$\begin{aligned}
\Delta w_{ij} &= \eta \left(\langle v_i \cdot h_j \rangle^M_{\text{DATA}} - \langle v_i \cdot h_j \rangle^M_{\text{RECON}} \right), \\
\Delta b_i &= \eta \left(\langle v_i \rangle^M_{\text{DATA}} - \langle v_i \rangle^M_{\text{RECON}} \right), \\
\Delta b_j &= \eta \left(\langle h_j \rangle^M_{\text{DATA}} - \langle h_j \rangle^M_{\text{RECON}} \right),
\end{aligned}$$

Donde η es la tasa de aprendizaje y las medias $\langle \cdot \rangle$ se calculan sobre las distribuciones correspondientes en los conjuntos *DATA* y *RECON*.

5.2.2 Evaluación

El proceso para evaluar la red es casi idéntico al proceso de entrenamiento, solo difiere que la probabilidad se evalúa directo en los valores de los pixeles de la imagen, siendo entonces la siguiente secuencia

$$
\begin{aligned}
&1) \quad m_i = \text{pixel}(i) \\
&2) \quad h_j = 1 \text{ con probabilidad } p = \sigma\left(\sum_i w_{ij} m_i + b_j\right) \text{ con } \sigma(x) = \frac{1}{1+e^{-x}} \\
&3) \quad \text{Guardar } (v_i, h_j) \text{ en el conjunto DATA} \\
&4) \quad m_i = \sum_i w_{ij} h_j + b_i
\end{aligned}
$$

5.2.3 La base de datos MNIST

Las imágenes utilizadas para el entrenamiento y evaluación de las RBM fueron obtenidas de la base de datos MNIST.

MNIST contiene imágenes de dígitos escritos a mano, del 0 al 9, con las siguientes características:

- **Formato de las imágenes:** Cada imagen es de tamaño 28×28 píxeles y está en escala de grises, con valores de intensidad en el rango de 0 (negro) a 255 (blanco).
- **Número de muestras:**
  - **Conjunto de entrenamiento:** 60,000 imágenes.
  - **Conjunto de prueba:** 10,000 imágenes.
- **Etiquetas:** Cada imagen está asociada con una etiqueta que indica el dígito correspondiente (del 0 al 9).

5.3 Implementación y resultados

Se implemento una Maquina de Boltzmann Restringida (RBM) y se la entrenó con el set de imágenes de la base de datos MNIST. Como las imágenes de la base de datos son de 28x28 pixeles, la capa visible de la RBM tiene 768 neuronas y se determino, empíricamente, que utilizando 70 neuronas en la capa oculta y una tasa de aprendizaje $\eta = 0.035$, se logra reconstruir los dígitos de la base de datos, alcanzando un mínimo del error en un tiempo aceptable de 3000 épocas. En la Fig. 20 y 21 se observan estos resultado obtenidos. Cabe aclarar que el valor del error no es representativo por la definición del error que se uso en donde no se dividía por la cantidad de imágenes utilizadas.

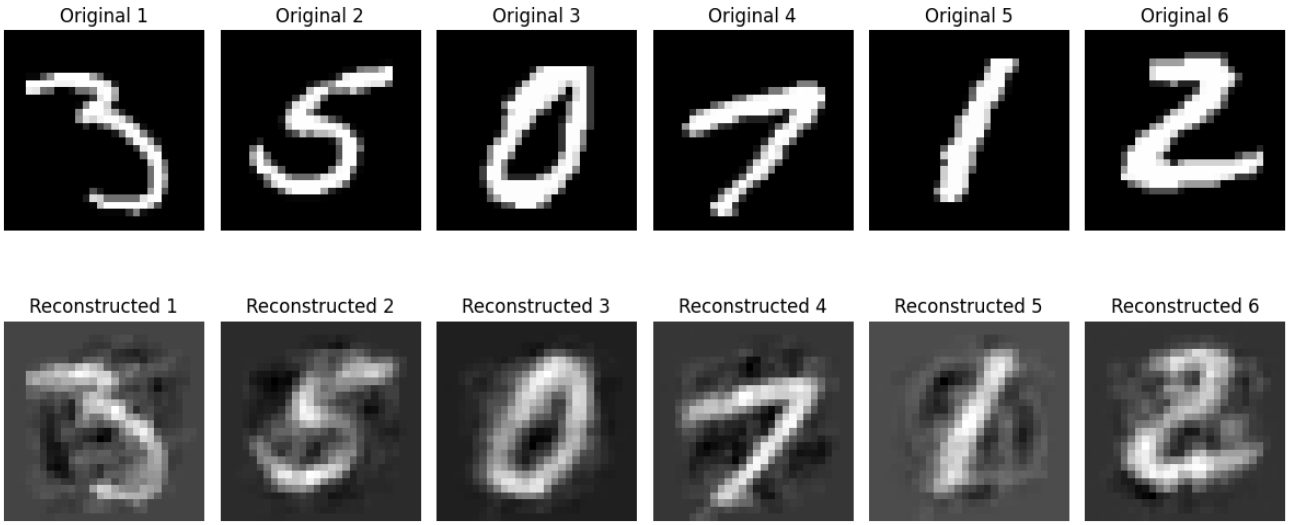


Figure 20: Imágenes originales y reconstrucción obtenida

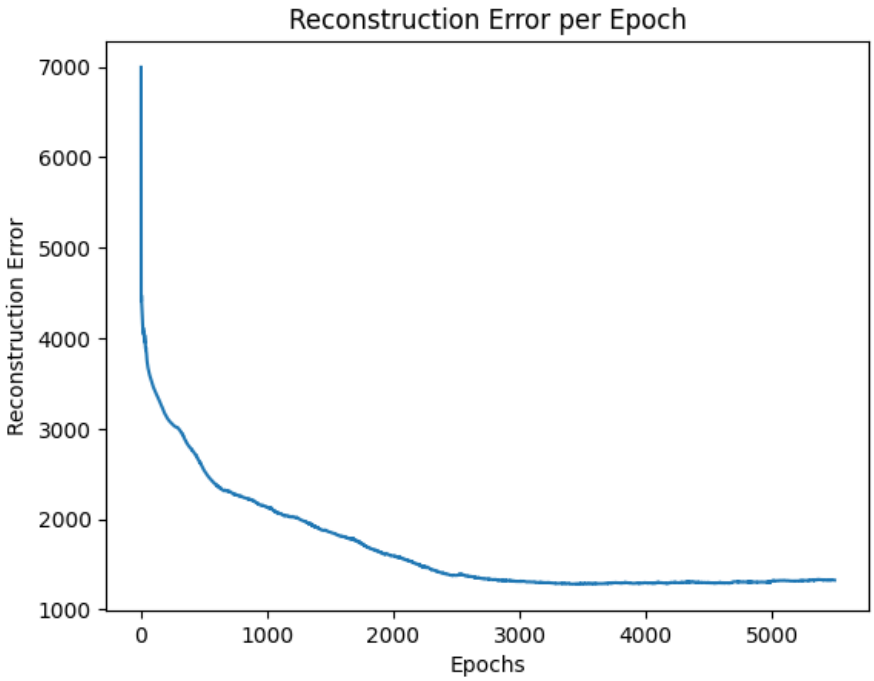


Figure 21: Evolución del error durante el entrenamiento

6 Ejercicio 6

6.1 Enunciado

Entrene una red convolucional para clasificar las imágenes de la base de datos MNIST. ¿Cuál es la red convolucional más pequeña que puede conseguir con una exactitud de al menos 90% en el conjunto de evaluación? ¿Cuál es el perceptrón multicapa más pequeño que puede conseguir con la misma exactitud?

6.2 Redes Neuronales Convolucionales (CNN)

Las redes neuronales convolucionales (CNN) son un tipo de red neuronal diseñada para procesar datos que tienen una estructura de tipo grilla, como las imágenes.

6.2.1 Estructura y Funcionamiento

Una CNN se compone principalmente de tres tipos de capas:

1. Capas convolucionales

Estas capas aplican filtros (*kernels*) a la entrada para extraer características locales, como bordes, texturas o formas. La operación de convolución consiste en un producto punto entre los valores del filtro y una región de la entrada, generando un mapa de características. Los filtros son parámetros aprendibles durante el entrenamiento, lo que permite a la red identificar patrones relevantes para la tarea específica.

2. Capas de agrupamiento (Pooling)

Estas capas reducen la dimensionalidad de los mapas de características, conservando las características más importantes.

- **Max pooling**: Selecciona el valor máximo en una región.
- **Average pooling**: Calcula el promedio en una región.

Al disminuir las dimensiones, las capas de pooling ayudan a reducir la complejidad computacional y a prevenir el sobreajuste.

3. Capas Lineales

Conectan todas las neuronas de una capa a la siguiente, como un perceptron multicapa. Estas capas combinan las características aprendidas en las etapas anteriores para realizar la clasificación o regresión.

6.2.2 Conceptos Clave

- **Stride (paso)**: Controla el desplazamiento del filtro sobre la entrada. Un stride mayor reduce las dimensiones de salida.
- **Padding (relleno)**: Agrega ceros alrededor de la entrada para mantener las dimensiones originales tras la convolución.
- **Funciones de activación**: La activación ReLU (*Rectified Linear Unit*) es común en CNN, ya que introduce no linealidades y mejora el aprendizaje.

6.3 Implementación y resultados

6.3.1 Red Neuronal Convolucional

Para clasificar imágenes del dataset MNIST se busco conseguir la red neuronal convolucional mínima. Como esta definición se mantiene a que la red tenga la menor cantidad de de parámetros, se implemento una red con la siguiente estructura:

- **Primera capa convolucional (conv1):**
  - Canales de entrada: 1, Canales de salida: 2, Tamaño del kernel: 3×3, Stride: 1, Padding: 1

- **Primera capa de MaxPooling:**
  - Tamaño del kernel: 2×2, Stride: 2
- **Segunda capa convolucional (conv2):**
  - Canales de entrada: 2, Canales de salida: 3, Tamaño del kernel: 7×7, Sin padding
- **Segunda capa de MaxPooling:**
  - Tamaño del kernel: 2×2, Stride: 2
- **Tercer MaxPooling:**
  - Tamaño del kernel: 2×2, Stride: 2
- **Capa completamente conectada (fc1):**
  - Neuronas de entrada: 12, Neuronas de salida: 10 (una para cada clase de dígito)

Luego se calcula la cantidad de parámetros como:

- **Primera capa convolucional (conv1):**

$$\text{Parámetros} = (3 \times 3 \times 1 \times 2) + 2 = 18 + 2 = 20$$

- **Segunda capa convolucional (conv2):**

$$\text{Parámetros} = (7 \times 7 \times 2 \times 3) + 3 = 294 + 3 = 297$$

- **Capa completamente conectada (fc1):**

$$\text{Parámetros} = (12 \times 10) + 10 = 120 + 10 = 130$$

- Cantidad total de parametros :

$$\text{Total} = 20\,(\text{conv1}) + 297\,(\text{conv2}) + 130\,(\text{fc1}) = 447\,\text{parámetros}$$

Tomando la exactitud como la cantidad de imágenes del set de evaluación se obtuvieron en 2 realizaciones de la experiencia. En ambos casos la red fue entrenada durante 7 épocas con un tamaño de Batch de 32 y una tasa de aprendizaje $\eta = 0.01$. En una de ellas, la exactitud fue del 92.78% del set de evaluación y en la otra fue del 92.73%.

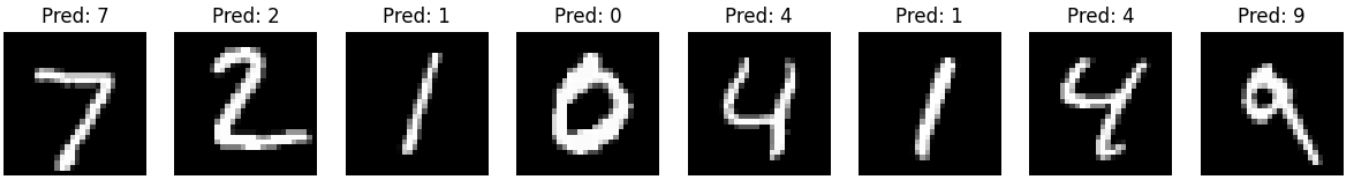


Figure 22: Predicción con exactitud del 92.78% - red convolucional mínima

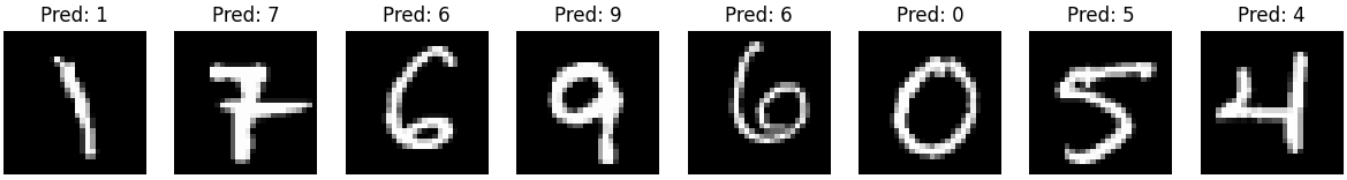


Figure 23: Predicción con exactitud del 92.73% - red convolucional mínima

6.3.2 Perceptrón multicapa

Para el caso del perceptron multicapa mínimo que puede aprender esta red se tiene que el modelo definido tiene una capa lineal (`nn.Linear`) con las siguientes características:

- Número de entradas: $28 \times 28 = 784$
- Número de salidas: 10

El número total de parámetros de la capa lineal es

$$\text{Parámetros totales} = (\text{número de entradas} + 1) \times \text{número de salidas} = (784 + 1) \times 10 = 7850 \text{ parámetros}$$

Finalmente, en 2 realizaciones donde el perceptrón multicapa se entreno durante 15 épocas con un tamaño de Batch de 32 y una tasa de aprendizaje $\eta = 0.001$ se obtuvieron una exactitud de 92.44% y 92.47%

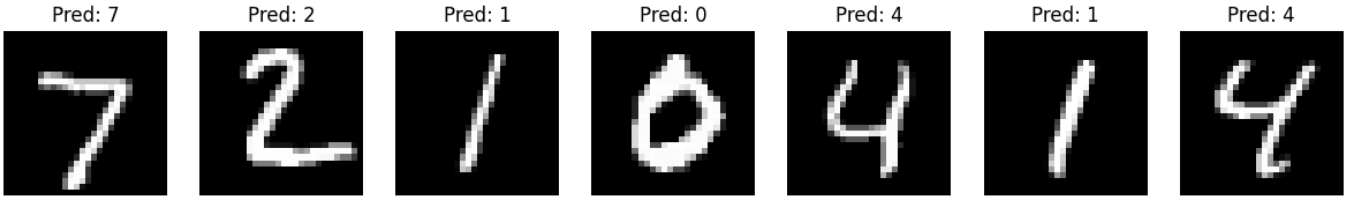


Figure 24: Predicción con exactitud del 92.44% - perceptrón multicapa mínimo

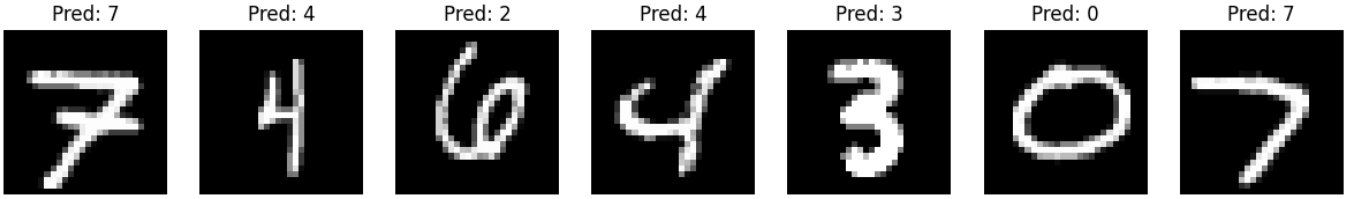


Figure 25: Predicción con exactitud del 92.47% - perceptrón multicapa mínimo

Se concluye que ambas redes son capaces de aprender y clasificar el set de imágenes MNIST pero las redes convolucionales logran hacerlo con una cantidad bastante menor de parámetros. Además, la red convolucional logro aprender y clasificar el set de imágenes MNIST con la misma precisión que el perceptrón multicapa entrenadose con la mitad de las épocas.

7 Ejercicio 7

7.1 Enunciado

Entrene un autoencoder para obtener una representación de baja dimensionalidad de las imágenes de MNIST. Use dichas representaciones para entrenar un perceptrón multicapa como clasificador. ¿Cuál es el tiempo de entrenamiento y la exactitud del clasificador obtenido cuando parte de la representación del autoencoder, en comparación con lo obtenido usando las imágenes originales?

7.2 Autoencoder

Un **autoencoder** es un tipo de red neuronal diseñado para aprender una representación comprimida (*codificación*) de los datos de entrada de manera no supervisada. Su estructura consiste en dos componentes principales: el **encoder** y el **decoder**.

- **Encoder (Codificador):** Reduce la dimensionalidad de los datos de entrada a un espacio más pequeño (*representación comprimida*). Por ejemplo, transforma una imagen de 28×28 píxeles en un vector de 10 dimensiones.
- **Decoder (Decodificador):** Reconstruye los datos originales a partir de la representación comprimida. El objetivo es que la salida reconstruida sea lo más parecida posible a la entrada original.

La red tiene una capa intermedia de menor dimensionalidad que obliga al modelo a aprender las características más importantes de los datos. La activación típica en el decoder es una función *sigmoide* (para restringir la salida entre 0 y 1 en tareas como imágenes normalizadas).

1. La entrada x pasa por el **encoder**, que genera una representación comprimida z.

$$z = \text{Encoder}(x)$$

2. z se utiliza para reconstruir los datos mediante el **decoder**.

$$\hat{x} = \text{Decoder}(z)$$

3. El autoencoder se entrena minimizando una función de pérdida, como el *Error Cuadrático Medio (MSE)*, que mide la diferencia entre la entrada x y la salida reconstruida $\hat{x}$.

7.3 Implementación y resultados

El objetivo de este experimento fue entrenar un **autoencoder** para obtener una representación de baja dimensionalidad de las imágenes del conjunto de datos **MNIST**. Posteriormente, se utilizó dicha representación comprimida para entrenar un **perceptrón multicapa (MLP)** como clasificador. Finalmente, se compararon los resultados en términos de tiempo de entrenamiento y exactitud, tanto para las representaciones comprimidas como para las imágenes originales.

1. **Autoencoder:** Se implementó un autoencoder con la siguiente arquitectura:
   - **Encoder:** Dos capas lineales que reducen las imágenes de 28×28 píxeles (784 dimensiones) a un espacio de 70 dimensiones y finalmente se vuelven a reducir a 40 dimensiones.
   - **Decoder:** Dos capas lineales que reconstruyen las imágenes a partir del espacio reducido.

   El modelo fue entrenado utilizando el *Error Cuadrático Medio (MSE)* como función de pérdida y un optimizador Adam con una tasa de aprendizaje de 0.001.

2. **Perceptrón Multicapa (MLP):** Se entrenaron dos MLPs:
   - **Con imágenes originales:** El modelo recibió como entrada las imágenes aplanadas (784 dimensiones).
   - **Con representaciones comprimidas:** El modelo utilizó como entrada las representaciones de 40 dimensiones generadas por el autoencoder.

   Ambos MLPs tuvieron una arquitectura de tres capas: una capa oculta con 30 neuronas, otra con 20, y una capa de salida con 10 neuronas (para las clases de dígitos).

3. **Entrenamiento y evaluación:** Cada modelo fue entrenado durante 5 épocas con un tamaño de batch de 32. Se midió el tiempo de entrenamiento y la precisión del clasificador en el conjunto de prueba.

Resultados

Los resultados obtenidos se resumen en la siguiente tabla:

Modelo	**Tiempo de entrenamiento (s)**	**Precisión (%)**
MLP con imágenes originales	10.5227	96.28
MLP con representaciones comprimidas	7.6044	92.62

Table 9: Comparación de tiempo de entrenamiento y precisión entre los dos clasificadores.

En la Fig. 26 y 27 se muestran los resultados de la evaluación del perceptrón multicapa cuando se lo evaluó con las imágenes comprimidas y cuando se lo evaluó con las imágenes originales de la base de datos de MNIST.

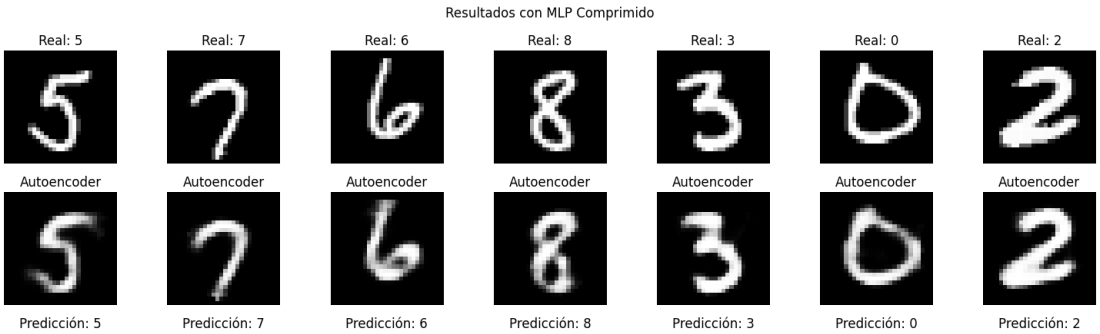


Figure 26: Predicción del perceptrón multicapa con imágenes comprimidas

Figure 27: Predicción del perceptrón multicapa con las imágenes originales

Tiempo de entrenamiento: El MLP entrenado con representaciones comprimidas presentó un tiempo de entrenamiento significativamente menor en comparación con el modelo que utilizó imágenes originales. Esto se debe a la menor dimensionalidad de las entradas, lo que reduce la cantidad de parámetros y operaciones necesarias durante el entrenamiento.

Precisión: La precisión del clasificador entrenado con representaciones comprimidas fue comparable (o ligeramente inferior) a la obtenida con imágenes originales. Esto indica que el autoencoder logró capturar características relevantes de las imágenes en un espacio de baja dimensionalidad. Se observa en la Fig. 26 que el perceptrón multicapa no fue capaz de predecir el ante ultimo 3 dado que su par comprimido se asemejaba mas a un 5, que fue lo que termino prediciendo el MLP.

En conclusión, el uso de un autoencoder para reducir la dimensionalidad de las imágenes de MNIST permite entrenar clasificadores MLP más rápido sin una pérdida significativa de precisión. Esta técnica es especialmente útil en escenarios donde los recursos computacionales son limitados o cuando se requiere acelerar el entrenamiento.